

Multiplicação matriz-vetor e o efeito da memória cache

Matheus Marinho

20 de agosto de 2026

1. Objetivo

O objetivo desta tarefa foi implementar duas versões da multiplicação de uma matriz por um vetor e comparar seus tempos de execução. As versões realizam o mesmo cálculo, mas percorrem a matriz em ordens diferentes: por linhas e por colunas.

2. Implementação

Para uma matriz quadrada A , um vetor x e um vetor de resultado y , cada posição do resultado é calculada por:

$$y[i] = A[i][0] \cdot x[0] + A[i][1] \cdot x[1] + \dots + A[i][N-1] \cdot x[N-1]$$

Versão	Laço externo	Laço interno	Ordem percorrida
Por linhas	linha i	coluna j	elementos consecutivos da linha
Por colunas	coluna j	linha i	elementos da mesma coluna

As duas versões foram executadas sobre os mesmos dados. O programa também compara os vetores produzidos para verificar se ambos os métodos chegaram ao mesmo resultado.

3. Medição do tempo

Foi utilizado o tempo de parede (*wall time*) por meio de `clock_gettime(CLOCK_MONOTONIC)`. Esse relógio mede o tempo transcorrido e não é afetado por ajustes no relógio do sistema.

Os testes menores são repetidos várias vezes e o tempo total é dividido pela quantidade de repetições. Antes da medição, cada versão é executada uma vez como aquecimento. Isso reduz a influência da resolução do cronômetro e da primeira alocação das páginas de memória.

O programa foi compilado com `gcc -O2 -Wall -Wextra -std=c99`. Os resultados são a mediana de três execuções em um Intel Core i7-13650HX com Windows 11. O tempo pode variar entre computadores e entre execuções.

4. Resultados

N	Matriz (MB)	Por linhas (s)	Por colunas (s)	Colunas ÷ linhas
64	0,03	0,000002	0,000002	0,98×
128	0,13	0,000010	0,000018	1,82×
256	0,50	0,000046	0,000103	2,25×
512	2,00	0,000193	0,001298	6,72×
1024	8,00	0,000793	0,005305	6,76×
2048	32,00	0,003330	0,023834	7,11×
4096	128,00	0,014773	0,107294	7,25×

Para $N = 64$, os tempos foram praticamente iguais. Em $N = 128$ e $N = 256$, a versão por colunas já apresentou alguma diferença, mas os tempos absolutos ainda eram muito pequenos. A divergência se tornou significativa em $N = 512$, quando o acesso por colunas passou a levar aproximadamente **6,7 vezes** o tempo do acesso por linhas. Nos tamanhos seguintes, a diferença permaneceu próxima ou superior a esse valor.

5. Explicação pela memória cache

Em C, os elementos de uma matriz são armazenados por linhas. Por isso, `matriz[i * n + j]` e `matriz[i * n + j + 1]` ocupam posições vizinhas na memória.

O processador não busca apenas um `double` por vez na memória principal. Ele traz um bloco chamado **linha de cache**, que contém vários elementos consecutivos. Na versão por linhas, esses elementos são usados logo nas próximas iterações.

Esse padrão sequencial também é fácil de antecipar pelo processador.

Na versão por colunas, cada incremento de i salta aproximadamente N elementos na memória. Assim, o processador traz uma linha de cache, utiliza apenas uma parte dela naquele momento e salta para uma região distante. Quando a matriz é pequena, grande parte dos dados permanece na cache e a diferença é pequena. Conforme a matriz cresce, os dados deixam de caber nas caches mais rápidas e o acesso por colunas provoca mais buscas em níveis mais lentos da memória.

O processador utilizado possui 24 MB de cache compartilhada. Em $N = 2048$, a matriz ocupa 32 MB e já ultrapassa essa capacidade. Isso ajuda a explicar por que a diferença permanece elevada nos maiores testes. O ponto exato da divergência não é universal: ele depende dos tamanhos das caches, do processador, do compilador e das dimensões testadas.

6. Conclusão

As duas versões executam a mesma multiplicação e produzem o mesmo resultado, mas possuem tempos diferentes devido ao padrão de acesso à memória. Nesta máquina, a divergência ficou clara a partir de $N = 512$. O acesso por linhas foi mais rápido porque percorreu endereços consecutivos e aproveitou melhor as linhas de cache. O experimento mostra que o desempenho não depende apenas da quantidade de operações: a ordem em que os dados são acessados também é importante.

7. Código-fonte

mxv.c · 152 linhas

```
1 /*
2  * Tarefa 2 - Multiplicacao de matriz por vetor
3  *
4  * Compilar:
5  * gcc -O2 -Wall -Wextra -std=c99 mxv.c -o mxv.exe -lm
6  */
7
8 #define _POSIX_C_SOURCE 199309L
9
10 #include <stdio.h>
11 #include <stdlib.h>
12 #include <math.h>
13 #include <time.h>
14
15 /* Impede que o compilador descarte chamadas repetidas durante a medicao. */
16 volatile double controle = 0.0;
17
18 double walltime(void)
19 {
20     struct timespec tempo;
21     clock_gettime(CLOCK_MONOTONIC, &tempo);
22     return tempo.tv_sec + tempo.tv_nsec / 1000000000.0;
23 }
24
25 /* Versao 1: o laço interno percorre as colunas de uma mesma linha. */
26 void multiplicar_por_linhas(const double *matriz, const double *vetor,
27                             double *resultado, int n)
28 {
29     int i, j;
30
31     for (i = 0; i < n; i++) {
32         double soma = 0.0;
33
34         for (j = 0; j < n; j++) {
35             soma += matriz[(size_t)i * n + j] * vetor[j];
36         }
37
38         resultado[i] = soma;
39     }
40 }
41
42 /* Versao 2: o laço interno percorre as linhas de uma mesma coluna. */
43 void multiplicar_por_colunas(const double *matriz, const double *vetor,
44                              double *resultado, int n)
45 {
46     int i, j;
47
48     for (i = 0; i < n; i++) {
49         resultado[i] = 0.0;
50     }
51
52     for (j = 0; j < n; j++) {
53         for (i = 0; i < n; i++) {
54             resultado[i] += matriz[(size_t)i * n + j] * vetor[j];
55         }
56     }
57 }
58
59 /* Repete os testes pequenos para que o tempo possa ser medido com estabilidade. */
60 int calcular_repeticoes(int n)
61 {
62     long long elementos = (long long)n * n;
63     int repeticoes = (int)(100000000LL / elementos);
64
65     if (repeticoes < 1) {
66         repeticoes = 1;
67     }
68     if (repeticoes > 10000) {
69         repeticoes = 10000;
70     }
71
72     return repeticoes;
73 }
74
75 int main(void)
76 {
77     int tamanhos[] = {64, 128, 256, 512, 1024, 2048, 4096};
78     int quantidade = 7;
79     int teste;
80
81     printf("%-6s %-12s %-7s %-12s %-12s %s\n",
82           "N", "Matriz (MB)", "Repet.", "Linhas (s)",
```

```

83     "Colunas (s)", "Razao");
84
85     for (teste = 0; teste < quantidade; teste++) {
86         int n = tamanhos[teste];
87         size_t elementos = (size_t)n * n;
88         int repeticoes = calcular_repeticoes(n);
89         double *matriz = malloc(elementos * sizeof(double));
90         double *vetor = malloc((size_t)n * sizeof(double));
91         double *resultado_linhas = malloc((size_t)n * sizeof(double));
92         double *resultado_colunas = malloc((size_t)n * sizeof(double));
93         double inicio, tempo_linhas, tempo_colunas;
94         double maior_diferenca = 0.0;
95         int i, r;
96
97         if (matriz == NULL || vetor == NULL || resultado_linhas == NULL ||
98             resultado_colunas == NULL) {
99             printf("Nao foi possivel alocar memoria para N = %d.\n", n);
100             free(matriz);
101             free(vetor);
102             free(resultado_linhas);
103             free(resultado_colunas);
104             return 1;
105         }
106
107         for (i = 0; i < (int)elementos; i++) {
108             matriz[i] = (i % 10 + 1) * 0.1;
109         }
110         for (i = 0; i < n; i++) {
111             vetor[i] = 1.0;
112         }
113
114         /* Aquecimento: executa uma vez antes de iniciar o cronometro. */
115         multiplicar_por_linhas(matriz, vetor, resultado_linhas, n);
116         multiplicar_por_colunas(matriz, vetor, resultado_colunas, n);
117
118         inicio = walltime();
119         for (r = 0; r < repeticoes; r++) {
120             multiplicar_por_linhas(matriz, vetor, resultado_linhas, n);
121             controle += resultado_linhas[r % n];
122         }
123         tempo_linhas = (walltime() - inicio) / repeticoes;
124
125         inicio = walltime();
126         for (r = 0; r < repeticoes; r++) {
127             multiplicar_por_colunas(matriz, vetor, resultado_colunas, n);
128             controle += resultado_colunas[r % n];
129         }
130         tempo_colunas = (walltime() - inicio) / repeticoes;
131
132         for (i = 0; i < n; i++) {
133             double diferenca = fabs(resultado_linhas[i] - resultado_colunas[i]);
134             if (diferenca > maior_diferenca) {
135                 maior_diferenca = diferenca;
136             }
137         }
138
139         printf("%-6d %-12.2f %-7d %-12.6f %-12.6f %-12.6f %.2fx%s\n",
140             n, elementos * sizeof(double) / (1024.0 * 1024.0),
141             repeticoes, tempo_linhas, tempo_colunas,
142             tempo_colunas / tempo_linhas,
143             maior_diferenca < 1e-9 ? "" : " RESULTADOS DIFERENTES");
144
145         free(matriz);
146         free(vetor);
147         free(resultado_linhas);
148         free(resultado_colunas);
149     }
150
151     return 0;
152 }

```